

TaskFlow MVP

42장 풀버전 · 화면 상태 · 에러 케이스 · 시스템 명세 · 서버·프론트 스펙

A

인트로

01-04

B

로그인·회원가입

05-09

C

팀 선택

10-14

D

칸반

15-22

E

채팅

23-27

F

모바일

28-30

G

시스템 명세

31-36

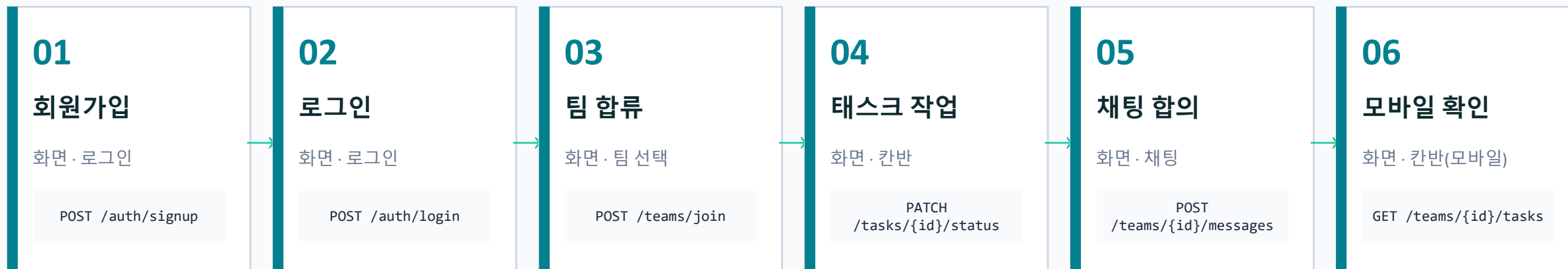
H~J

에러·스펙·마무리

37-42

사용자 여정 — 전체 흐름

신규 합류자 기준 6단계. 페르소나별 진입점은 다음 슬라이드에서 분기



주요 분기 (메인 흐름 외)

STEP 2 → 팀 선택

users.team_id = NULL → 강제 진입

STEP 3 → 팀 만들기

리더는 초대코드 발급 분기

STEP * → 401 로그인

JWT 만료 → 자동 redirect

STEP * → 403 에러

비멤버가 /teams/{id} 접근

페르소나 3종 — 진입 경로 분기

같은 시스템도 페르소나마다 첫 화면·체류 시간·핵심 행동이 다름

팀 리더

3-5인 팀 운영, 진행 파악·우선순위 조정

진입 화면 순서

1. 로그인
2. 팀 만들기
3. 초대코드 공유
4. 칸반(전체)
5. 채팅

핵심 행동	한 화면 대시보드
-------	-----------

체류 시간	체류 ~30분/세션
-------	------------

최다 API	GET /teams/{id}/tasks
--------	-----------------------

팀원

내 태스크 빠른 확인 → 드래그 → 짧은 채팅

진입 화면 순서

1. 로그인
2. 칸반(필터)
3. 채팅

핵심 행동	내 카드(@me) 중심
-------	--------------

체류 시간	체류 ~5분/회, 여러 번
-------	----------------

최다 API	PATCH /tasks/{id}/status
--------	--------------------------

신규 합류자

초대받음 → 가입 → 컨텍스트 1분 파악

진입 화면 순서

1. 회원가입
2. 초대코드 입력
3. 칸반(스크롤)
4. 채팅(이력)

핵심 행동	이력 시간순 스크롤 (검색 x)
-------	-------------------

체류 시간	첫 세션 ~10분
-------	-----------

최다 API	GET /teams/{id}/messages
--------	--------------------------

결정 추적표 — PDF 원본 → 통합본 변경 8건

1장 PDF의 빈틈을 메운 결정 8개. 본 스토리보드는 이 결정 기준

#	항목	PDF 원본	통합본 결정	이유
1	users-teams 멤버십	owner_id만 있음 (멤버 누가?)	users.team_id 추가 (1인 1팀)	DB 4테이블 제약 유지
2	신규 합류자 행동	채팅 이력 '검색'	시간순 '스크롤'로 변경	범위 외(검색)와 모순 해소
3	PUT /tasks/{id} 중복	status용·title용 2개 같은 path	PATCH /tasks/{id}/status 분리	REST 의미 + 칸반 드래그 별도
4	'내 태스크' 정의	creator_id만 있음	tasks.assignee_id 추가 (nullable)	내가 만든≠내 태스크
5	logout 의미	stateless인데 endpoint 존재	200만 반환 (블랙리스트 x)	범위 외 명시
6	권한 검증	admin/member 구분만	비멤버 403, DELETE 본인만	ACME Constraints 보강
7	측정 불가 NFR	드래그 50ms·1분 파악	정성 검증으로 명시	수동 동작 확인
8	GET /messages/{id}	용도 모호 (padding)	제거 → GET /teams/{id}로 교체	API 18개 유지

회원가입 — 초기 / 입력 중 / 처리 중

3가지 상태를 가로로 비교. validation은 클라이언트 + 서버 양쪽

초기 상태

taskflow.vercel.app

TaskFlow

회원가입

이메일 입력

비밀번호 (8자 이상)

가입하기

이미 계정이 있으신가요? 로그인

입력 중

taskflow.vercel.app

TaskFlow

회원가입

user@example.com

.....

가입하기

이미 계정이 있으신가요? 로그인

처리 중 (loading)

taskflow.vercel.app

TaskFlow

회원가입

user@example.com

.....

처리 중...

이미 계정이 있으신가요? 로그인

① email type=email 검증 · ② password 8자 이상 (클라이언트) · ③ POST /auth/signup → 201 + JWT 받기

회원가입 에러 — validation 실패 케이스

이메일 형식·이메일 중복·비밀번호 약함 — 각 케이스별 에러 표시

이메일 형식 오류 (400)

TaskFlow

회원가입

user@invalid

△ 올바른 이메일 형식이 아닙니다

.....

가입하기

이미 계정이 있으신가요? 로그인

이메일 중복 (409)

TaskFlow

회원가입

taken@example.com

.....

× 이미 가입된 이메일입니다

가입하기

이미 계정이 있으신가요? 로그인

비밀번호 약함 (400)

TaskFlow

회원가입

user@example.com

...

△ 8자 이상 입력해주세요

가입하기

이미 계정이 있으신가요? 로그인

에러 응답 표준: { error: { code: 'EMAIL_TAKEN', message: '이미 가입된 이메일입니다' } } · HTTP 4xx

로그인 — 정상 흐름

이메일 + 비밀번호 → 200 + JWT (24h 유효) → localStorage 저장 → 분기

The screenshot shows a web browser window with the URL 'taskflow.vercel.app'. The page has a teal header with the 'TaskFlow' logo. Below the logo is the title '로그인' (Login). There are two input fields: the first contains 'user@example.com' and the second contains masked characters '.....'. Below the input fields is a large teal button with the text '✓ 성공! 이동 중...' (Success! Moving...). At the bottom of the page, there is a link that says '계정이 없으신가요? 회원가입' (Don't have an account? Sign up).

응답 흐름

- 1 POST /auth/login
{ email, password }
- 2 → 200 OK + JWT (exp 24h)
localStorage.setItem('token', jwt)
- 3 → users.team_id 검사:
NULL → 팀 선택, ELSE → 칸반

응답 JSON 예시

```
{
  "token": "eyJhbGc...",
  "user": {
    "id": 42,
    "email": "user@example.com",
    "team_id": null
  }
}
```

로그인 401 실패 — 자격 증명 오류

이메일 존재 여부 노출 금지. 'INVALID_CREDENTIALS' 한 메시지로 통일

taskflow.vercel.app

TaskFlow

로그인

user@example.com

...

× 이메일 또는 비밀번호가 일치하지 않습니다

로그인

계정이 없으신가요? 회원가입

보안 고려사항

- 1 이메일 존재 여부 노출 금지
존재해도/없어도 동일 메시지
- 2 5회 연속 실패 시 60초 cooldown
(Day 2 범위 외 — 메모)
- 3 응답 시간 일정하게 (timing attack)

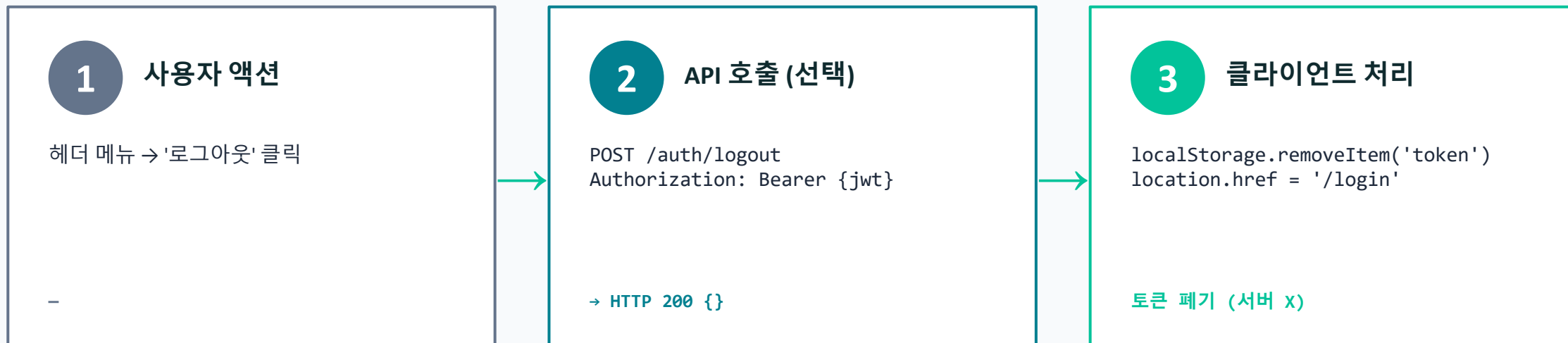
응답 JSON 예시

HTTP 401

```
{  "error": {    "code": "INVALID_CREDENTIALS",    "message": "이메일 또는 비밀번호가 일치하지 않습니다"  }}
```


로그아웃 — stateless 흐름

결정 #5: JWT 블랙리스트 없음. 클라이언트가 토큰 삭제, 서버는 200만 반환



⚠ 결정 #5: stateless 로그아웃

선택: JWT는 stateless이므로 서버에 블랙리스트를 유지하지 않음. 클라이언트가 토큰만 폐기.

근거: PDF Constraint '갱신 기능은 Day 2 범위 외' + 단순성 우선.

트레이드오프: 토큰 만료(24h) 전까지 유효함을 감수. 분실/탈취 시 강제 만료 불가.

팀 미가입 진입 — team_id = NULL

로그인 직후 분기점. team_id가 NULL인 사용자는 이 화면을 통과해야만 칸반 가능

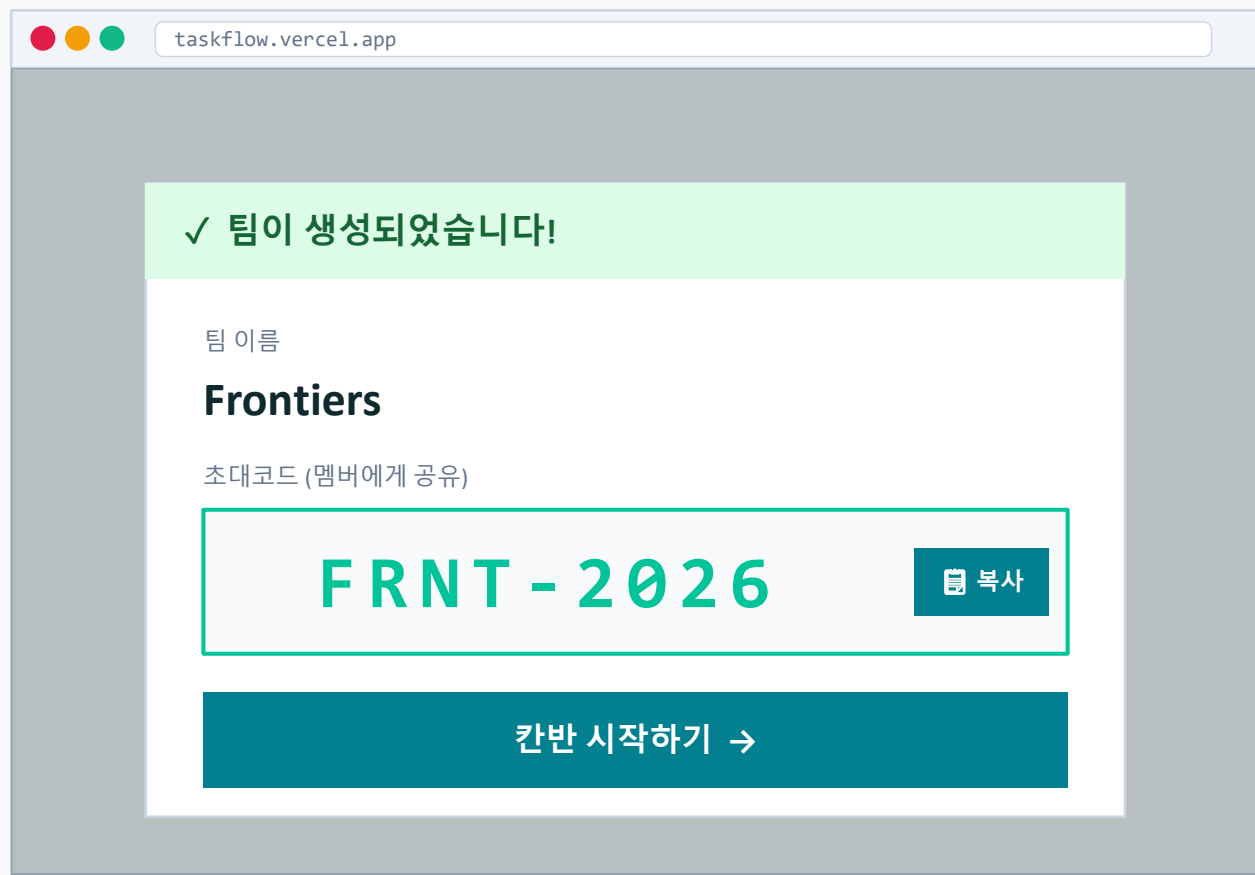
The screenshot shows a web browser window with the URL 'taskflow.vercel.app'. The page header includes the 'TaskFlow' logo and a user profile 'user@example.com | 로그아웃'. A light blue notification box states: '아직 팀에 소속되지 않았습니다. 팀을 만들거나 초대코드로 합류하세요.' Below this, there are two main sections: '+ 새 팀 만들기' and '초대코드로 합류'. The first section has a text input for '팀 이름 (1-30자)' and a '만들기' button. The second section has a text input showing 'ABCD-1234' and a '합류' button. A note below the code input specifies the format: '형식: 대문자 4 + 숫자 4 (하이픈 포함)'.

분기 로직

- 1 조건부 표시 — users.team_id가 NULL인 사용자에게만 이 화면 노출
- 2 이미 팀 보유 시 — 자동 redirect → /teams/{my_team_id} (칸반)
- 3 URL 직접 입력 차단 — /teams/* 에 접근 시 team_id 검증 → NULL이면 여기로 redirect
- 4 ESC/뒤로가기 시 — 로그인 화면 x
여기 머무름 (탈출 경로: 팀 가입만)
- 5 로그아웃 — 가능 (헤더 우측)

팀 만들기 → 초대코드 발급

생성자가 자동으로 owner_id가 되고, 초대코드는 서버가 자동 생성



시퀀스

- 1 POST /teams { name: 'Frontiers' }
- 2 → 서버: invite_code 자동 생성 (대문자 4 + '-' + 숫자 4)
- 3 → DB: teams INSERT (owner_id=me)
users.team_id UPDATE

응답 JSON

HTTP 201 Created

```
{
  "id": 7,
  "name": "Frontiers",
  "invite_code": "FRNT-2026",
  "owner_id": 42,
  "created_at": "2026-05-13T14:30:00Z"
}
```

초대코드 합류 — 정상 흐름

코드 입력 → 검증 → users.team_id UPDATE → 칸반으로 redirect

taskflow.vercel.app

초대코드 입력

FRNT - 2026

✓ Frontiers 팀이 확인되었습니다

팀 정보 (미리보기)

Frontiers

멤버 3명 · 태스크 12개 · owner: leader@ex.com

이 팀에 합류 →

검증 시퀀스

- 1 POST /teams/join { invite_code }
- 2 → 코드 유효성 검증 (형식 + 존재)
- 3 → users.team_id UPDATE = teams.id
응답에 팀 정보 포함

응답 JSON

```
HTTP 200 OK

{
  "team": {
    "id": 7,
    "name": "Frontiers",
    "member_count": 3
  },
  "redirect": "/teams/7"
}
```

초대코드 에러 — 형식·존재·중복 합류

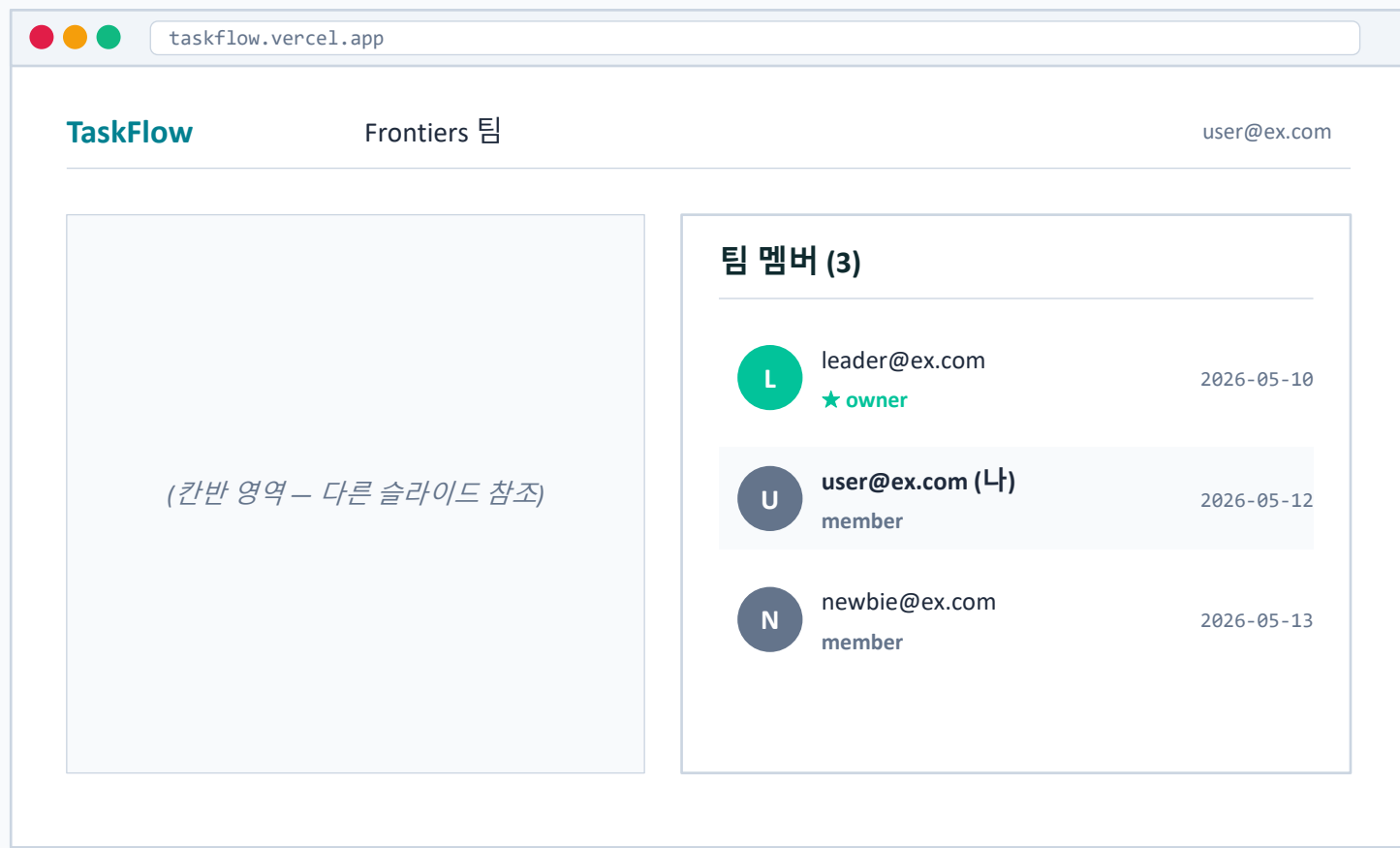
3가지 실패 케이스 시각화. 인라인 에러 + 가이드 문구

형식 오류 (400)	존재하지 않음 (404)	이미 다른 팀 소속 (409)
taskflow.vercel.app 초대코드 abcd1234 ⚠ 형식이 올바르지 않습니다 i 대문자 4 + 숫자 4 (예: FRNT-2026) 다시 입력	taskflow.vercel.app 초대코드 XXXX-9999 ⚠ 해당 초대코드를 찾을 수 없습니다 i 팀 리더에게 코드를 다시 확인하세요 다시 입력	taskflow.vercel.app 초대코드 FRNT-2026 ⚠ 이미 다른 팀에 소속되어 있습니다 i 기존 팀을 떠나야 합류 가능 (범위 외) 다시 입력

HTTP code 일관: 400 형식 / 404 미존재 / 409 충돌 · 응답 body { error: { code, message } }

팀 멤버 목록 — 사이드 패널

owner(★)와 member 구분만. 추방·역할 변경 범위 외

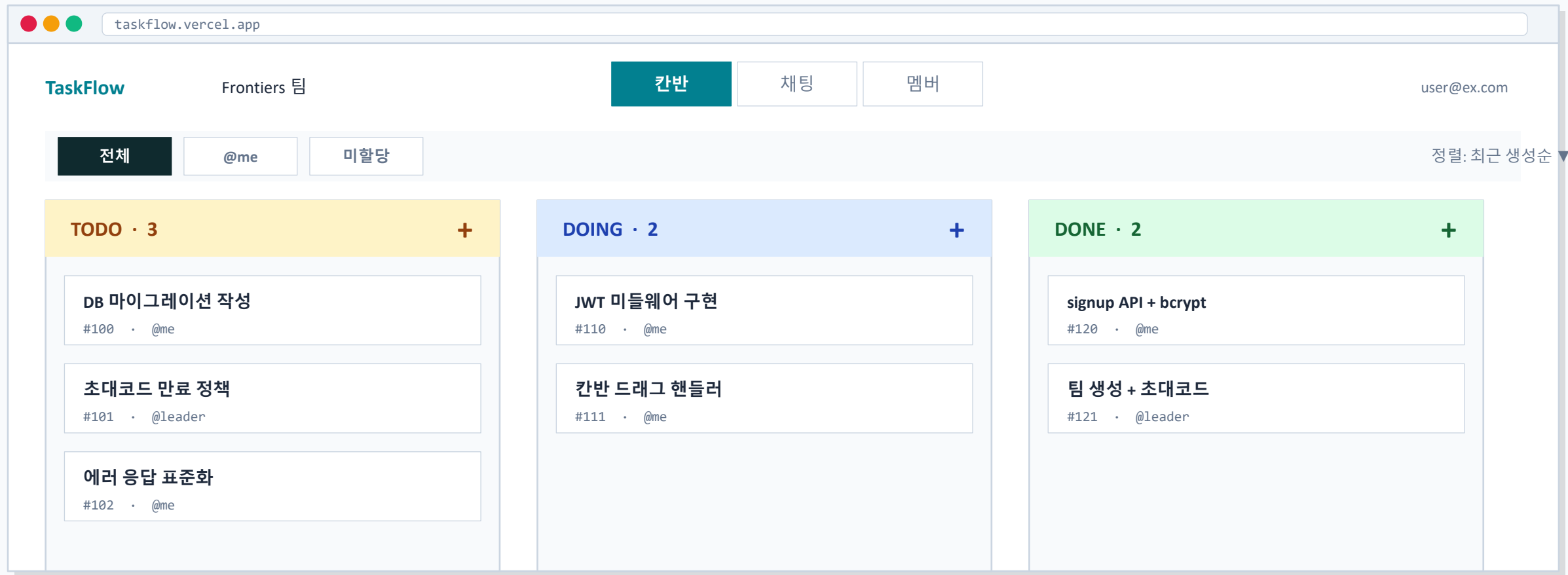


권한 모델 (결정 #6)

- 1 owner (1명, team.owner_id)
 - 팀 생성자 자동 지정
 - DELETE /tasks 가능 (본인 외 카드도)
- 2 member (N명)
 - 초대코드로 합류한 사용자
 - 본인 카드/메시지만 DELETE
- 3 비멤버 (다른 팀 또는 미가입)
 - GET /teams/{id}/* → 403
 - 모든 쓰기 API → 403
- 4 추방·역할 변경 — Day 2 범위 외

칸반 정상 상태 — 3컬럼 카드 가득

메인 작업 공간. TODO → DOING → DONE 카드 표시 + 필터 + 정렬



① 필터: 전체 / @me (내 카드) / 미할당

② 정렬: 최근 생성순 (tasks.created_at desc)

③ 카드 = 제목 + #id + @assignee

④ + 버튼 → 인라인 입력 (다음 슬라이드)

빈 칸반 — empty state

신규 팀 생성 직후 또는 모든 태스크 삭제 후, 첫 카드 추가 유도



i empty state는 신규 팀 + 첫 진입 시 + 모든 카드 삭제 후 3가지 시점에 표시. TODO 컬럼에만 CTA를 강조해서 가이드.

태스크 생성 — 인라인 입력

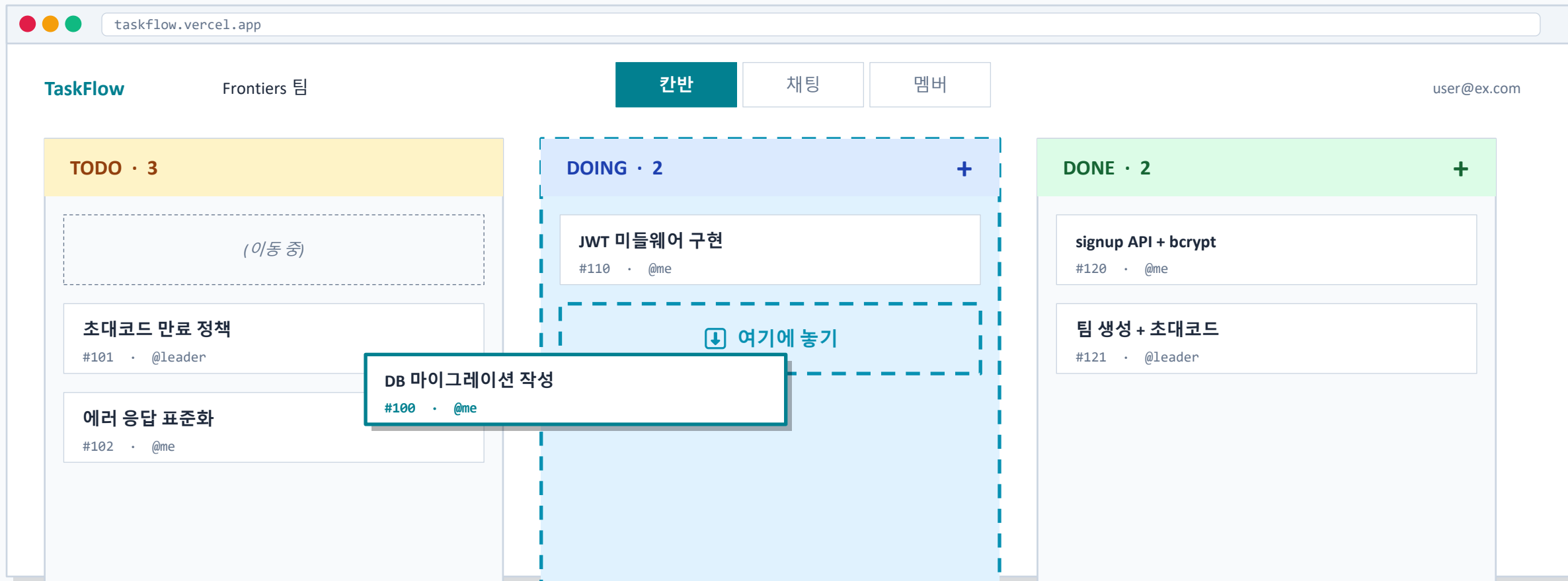
TODO 컬럼 헤더 '+' 클릭 → 인라인 입력란 등장 → Enter로 저장

The screenshot shows the TaskFlow application interface. At the top, there's a navigation bar with the TaskFlow logo, the team name 'Frontiers 팀', and buttons for '칸반' (Kanban), '채팅' (Chat), and '멤버' (Members). The user is logged in as 'user@ex.com'. The main area displays a Kanban board with three columns: 'TODO · 3 → 4' (highlighted in yellow), 'DOING · 2' (light blue), and 'DONE · 2' (light green). A modal is open for adding a task to the TODO column. The modal has a title '팀 멤버 페이지 페이지네이션' and a dropdown menu for the assignee, currently showing '@me'. Below the dropdown, it says 'Enter: 저장 · Esc: 취소'. There are two task cards listed: 'DB 마이그레이션 작성' (#100, assignee @me) and '초대코드 만료 정책' (#101, assignee @leader). The DOING and DONE columns contain placeholder text: '(생략 — 다른 슬라이드 참조)'.

시퀀스: [+ 클릭] → [인라인 입력 등장] → [제목 + assignee 입력] → [Enter] → POST /teams/{id}/tasks → 201 Created → 카드로 변환

카드 드래그 중 — TODO → DOING

드래그 시작 시 카드 unfocus, 대상 컬럼 highlight, drop 시 PATCH 호출



Drop 시: `PATCH /tasks/100/status { status: 'DOING' }` → 200 OK → 카드 새 위치 확정 (시각 반응 < 50ms 목표 – 정성 검증)

카드 상세 / 수정 모달

카드 클릭 → 모달 열림. 제목·상태·assignee 수정 + 메타 정보

taskflow.vercel.app

#110 JWT 미들웨어 구현

×

상태

TODO

DOING

DONE

저장

담당자

@me (user@ex.com)

다른 담당자 지정

생성자

@leader (leader@ex.com)

삭제

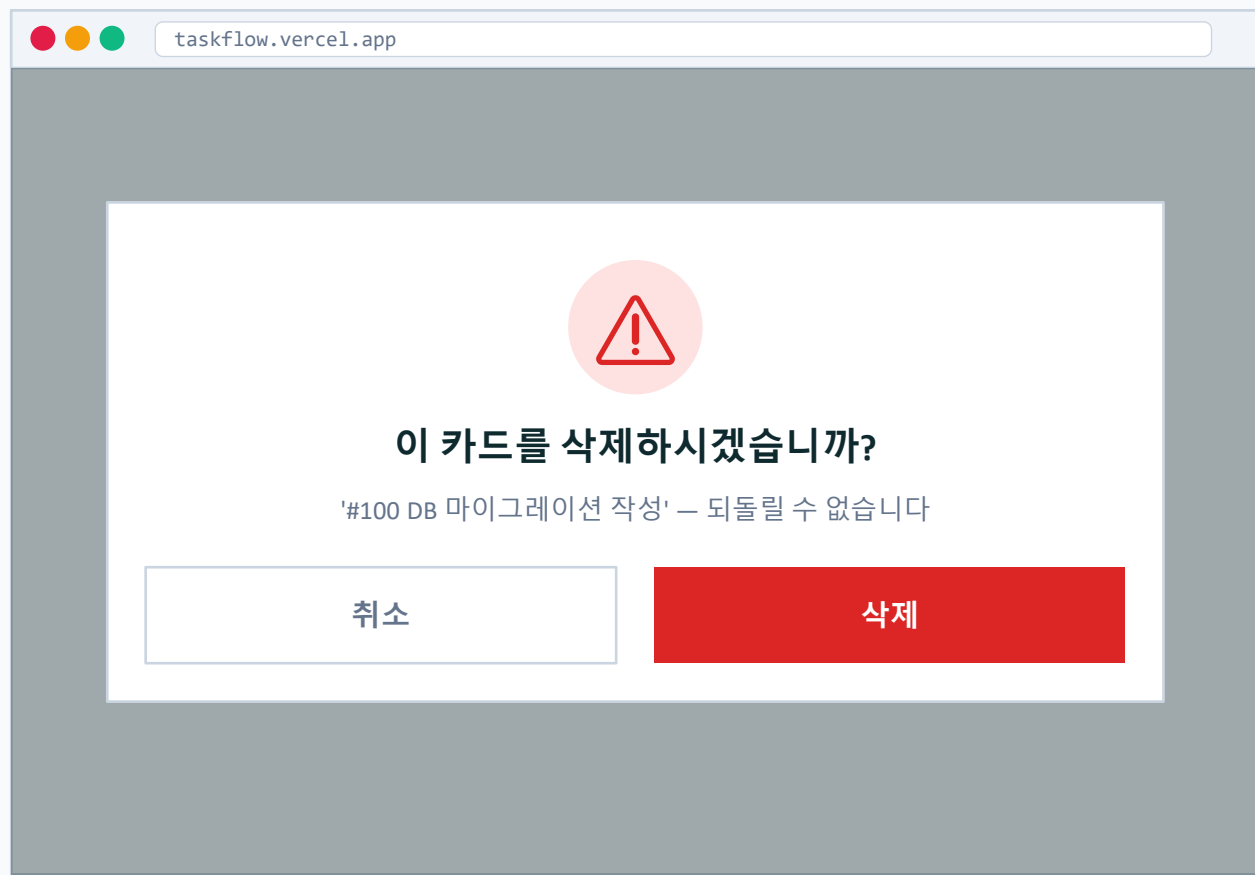
생성 시각

2026-05-12 14:30

PUT /tasks/110 { title } · PATCH /tasks/110/status { status } · DELETE /tasks/110

카드 삭제 — 확인 다이얼로그 + 권한

권한 규칙: creator(생성자) 또는 team owner만 삭제 가능. 비권한자는 버튼 숨김



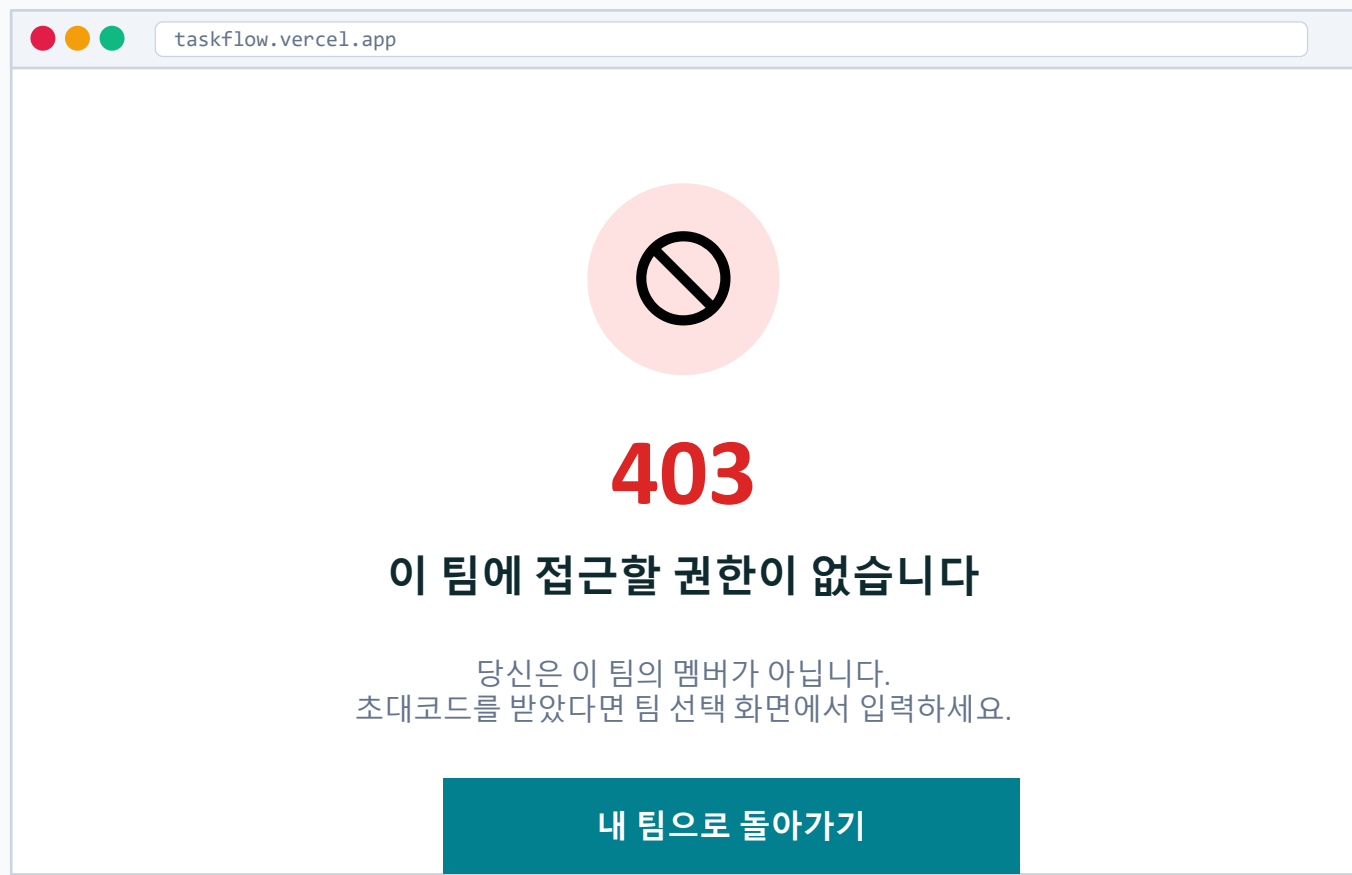
DELETE 권한 매트릭스

역할	본인 카드	타인 카드	메시지
owner	✓	✓ (오버라이드)	본인만
member (creator)	✓	✗	본인만
member (others)	✗	✗	본인만
비멤버	—	—	403

403 응답: { error: { code: 'FORBIDDEN', message: '권한이 없습니다' } }

비멤버 접근 — 403 Forbidden

다른 팀의 /teams/{id} URL을 직접 입력한 경우. 결정 #6의 시각화



트리거 조건

- 1 GET /teams/{other_id}/* 직접 호출 (브라우저 URL 입력 등)
- 2 유효한 JWT 있지만 user.team_id ≠ {id} (또는 미가입 상태)
- 3 POST/PATCH/DELETE도 동일하게 차단

응답 JSON

HTTP 403 Forbidden

```
{
  "error": {
    "code": "FORBIDDEN",
    "message": "이 팀의 멤버가\n아닙니다"
  }
}
```

assignee 선택 — nullable + 필터

결정 #4: tasks.assignee_id (nullable). 미할당 카드는 '미할당' 배지로 시각화

taskflow.vercel.app

담당자 선택

- ☒ @me (나)
user@ex.com
- ☐ @leader
leader@ex.com
- ☐ @newbie
newbie@ex.com
- ☐ 미할당 (null)
담당자 없음 — 누구나 가져갈 수 있음

미할당 카드 표시 + 필터

리뷰 PR #43 검토

#103 · ⚠ 미할당

필터 옵션 (칸반 상단)

전체 WHERE team_id = ?

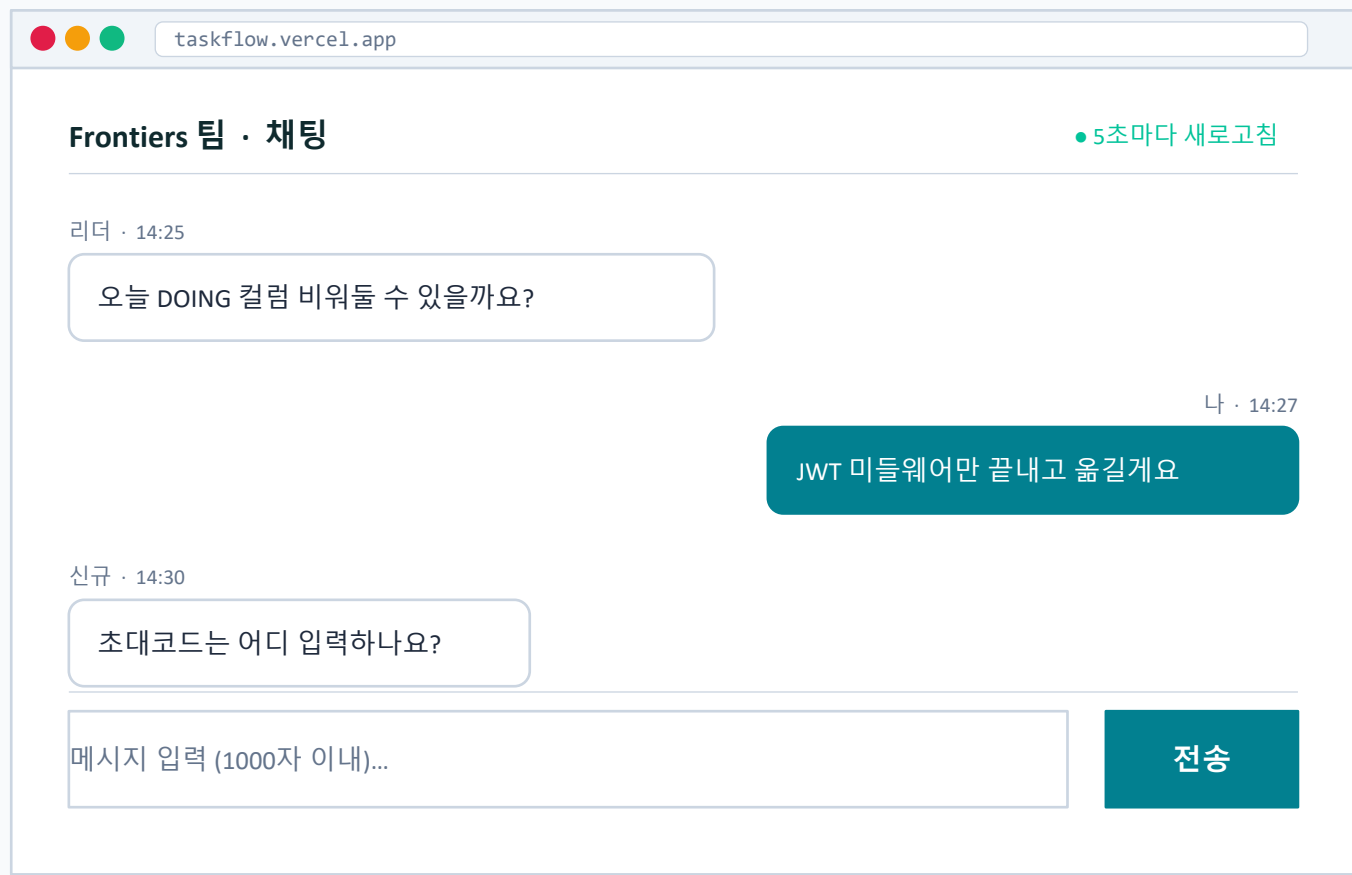
@me AND assignee_id = current_user_id

미할당 AND assignee_id IS NULL

결정 #4: assignee_id Nullable. '내 태스크' = WHERE assignee_id = current_user_id (creator 아님)

채팅 정상 — 말풍선 + 폴링

팀 단위 채팅. 5초 폴링으로 since= 파라미터 사용 → 새 메시지만 가져옴



폴링 로직

- 1 처음 진입: GET .../messages
(since 없음 → 최근 50개)
- 2 5초 후: GET .../messages?since=14:30:00
(마지막 메시지 시각)
- 3 응답이 빈 배열이면 화면 변화 없음

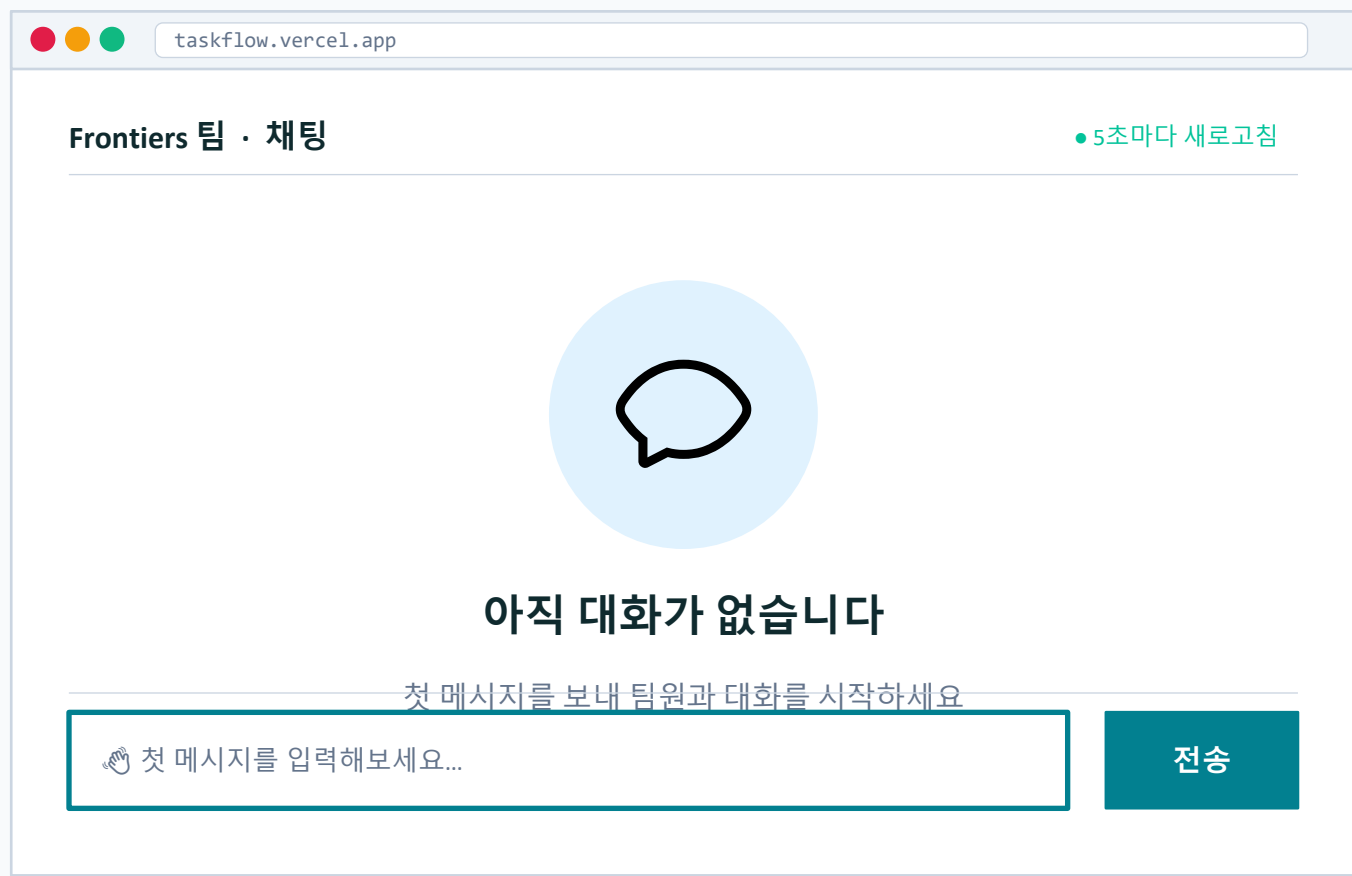
응답 JSON

HTTP 200 OK

```
[
  {
    "id": 23,
    "user_id": 42,
    "user_email": "user@ex.com",
    "content": "JWT 미들...",
    "created_at": "2026-05-13T14:27:00Z"
  }
]
```

빈 채팅 — empty state

신규 팀에 메시지가 하나도 없을 때. 첫 메시지 작성 유도



표시 조건 & 동작

- 1 조건 — 해당 team_id의 messages가 0건일 때 (DELETE 무관)
- 2 GET .../messages → [] 반환
- 3 사용자 메시지 1건 전송 즉시 → 일반 채팅 화면으로 전환
- 4 DELETE로 0건이 되어도 이 화면 재진입 (이력은 사라짐, 정책 x)
- 5 팀 합류 직후 신규 멤버에게도 노출 (페르소나 '신규 합류자' 첫 화면)

메시지 1000자 초과 — 카운터 적색

PDF Constraint: 1회당 1000자 이내. 클라이언트에서 카운터 + 서버에서 재검증

taskflow.vercel.app

Frontiers 팀 · 채팅

5초마다 새로고침

(이전 메시지 영역 — 다른 슬라이드 참조)

메시지 입력

어제 미팅에서 논의한 칸반 컬럼 분리 안에 대해서 좀 더 자세히 정리해서 공유드립니다. 우선 TODO/DOING/DONE 3컬럼은 유지하되, 각 컬럼 내에서 우선순위 라벨을 추가하는 게 어떨까 싶었습니다. 다만 이건 Day 2 범위에 포함되어 있지 않으니 다음 스프린트로 미루는 게 좋을 것 같고요...

1,042 / 1,00042자 초과

전송 (불가)

검증 정책 (2중)

- 1 클라이언트: 입력 시마다 length 체크
→ 1000 초과 시 카운터 적색 + 버튼 disable
- 2 서버: POST 요청 검증 → 400 Bad Request
→ { code: 'TOO_LONG', limit: 1000 }
- 3 두 곳에서 모두 검증하는 이유: API 직접 호출 차단

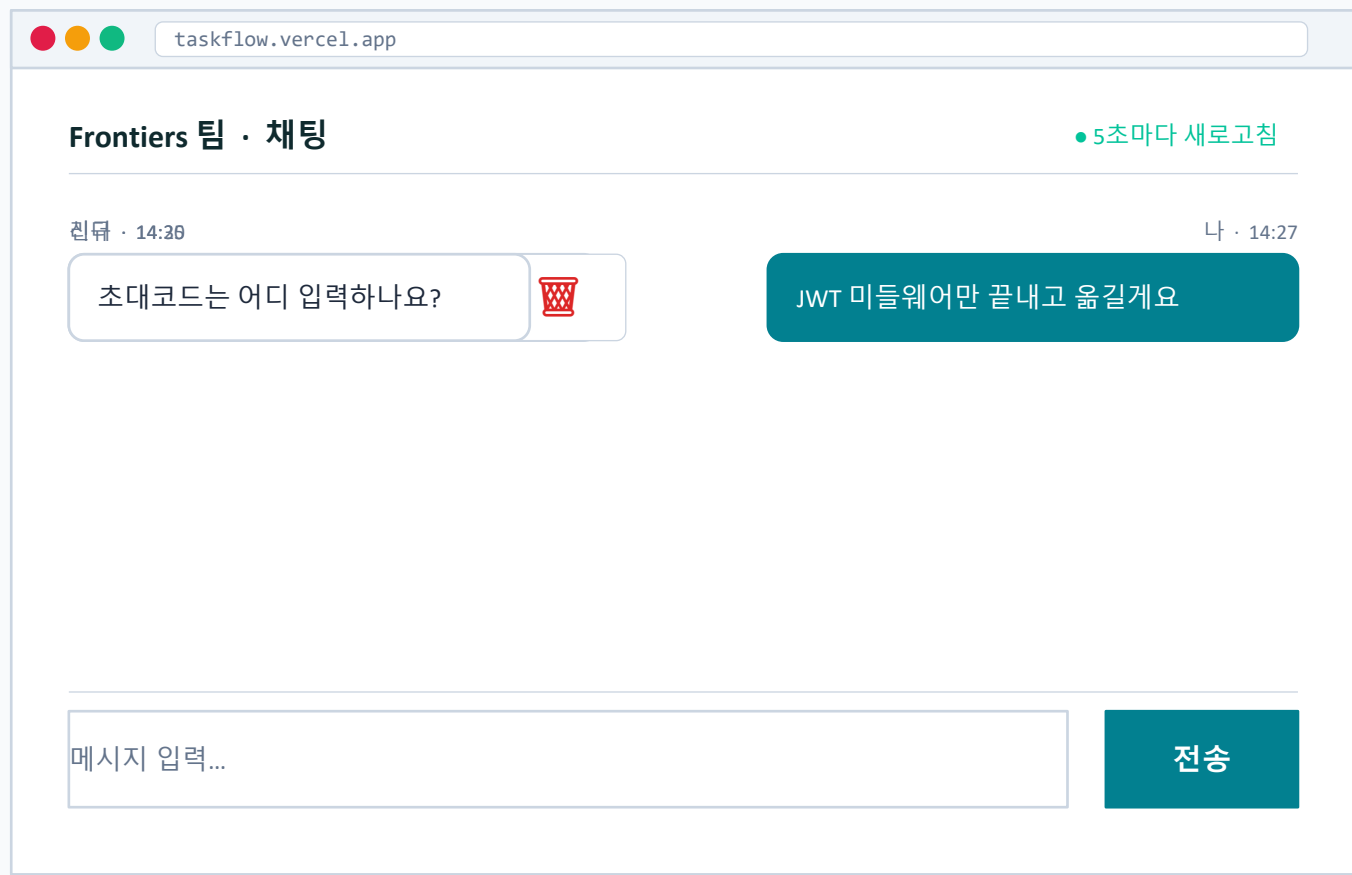
400 응답 JSON

HTTP 400 Bad Request

```
{  "error": {    "code": "TOO_LONG",    "message": "메시지는 1000자 이내",    "limit": 1000,    "actual": 1042  }}
```

메시지 삭제 — 본인만 가능 (호버 메뉴)

결정 #6: DELETE /messages/{id}는 본인 메시지만. owner도 타인 메시지 삭제 불가



삭제 권한 (결정 #6)

- 1 본인 메시지: 🗑️ 아이콘 호버 시 표시
클릭 시 즉시 삭제 (확인 x)
- 2 타인 메시지: 아이콘 자체 없음
API 직접 호출 시 403
- 3 owner여도 타인 메시지는 삭제 불가
(커뮤니티 신뢰 모델)

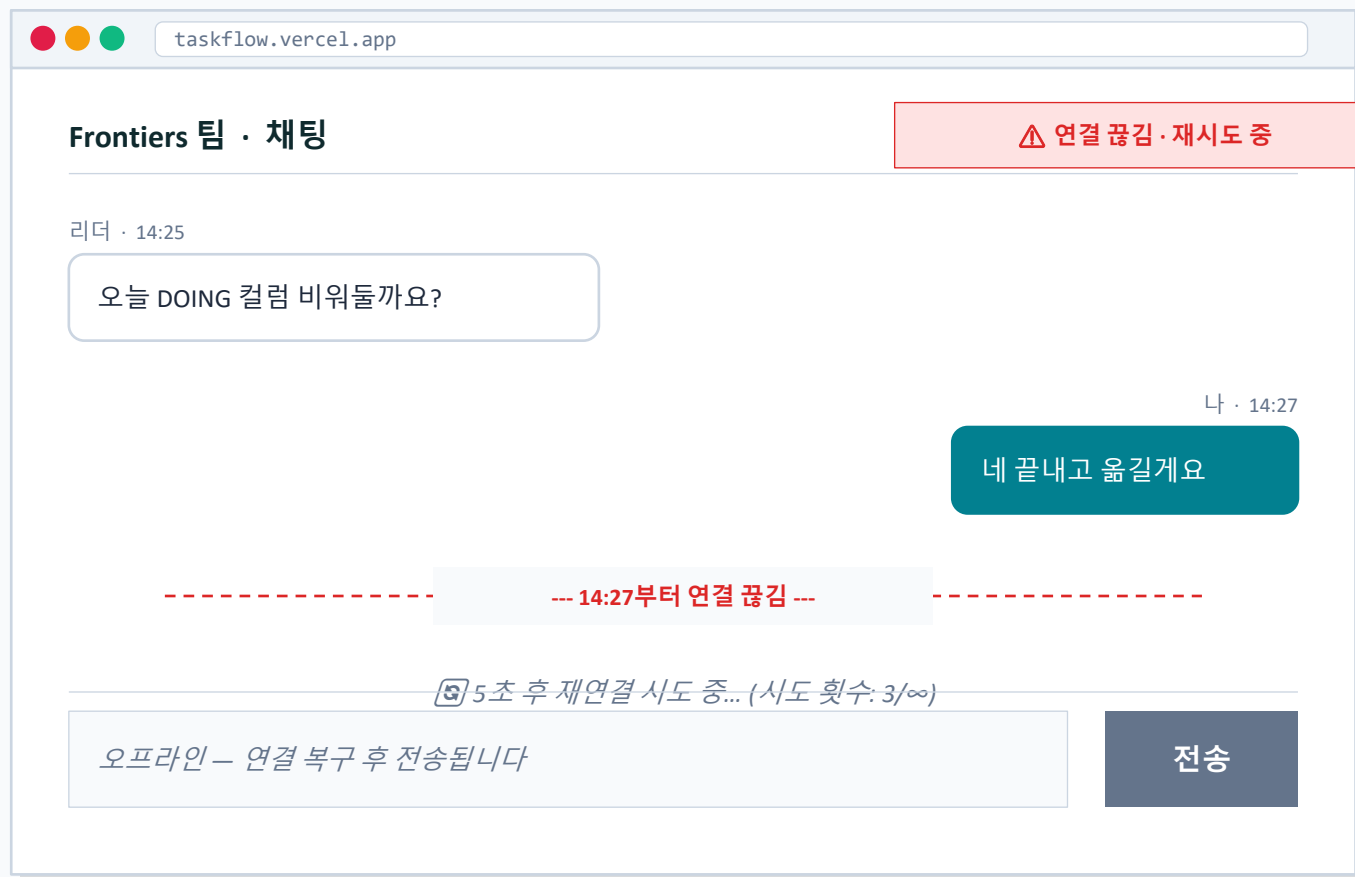
403 응답 JSON

```
DELETE /messages/45
HTTP 403 Forbidden

{
  "error": {
    "code": "NOT_OWNER",
    "message": "본인 메시지만  
삭제 가능"
  }
}
```

폴링 실패 — 네트워크 끊김 표시

5초 폴링 중 실패 시 재시도 + 사용자에게 알림. 메시지 누락 0건 보장



재시도 로직

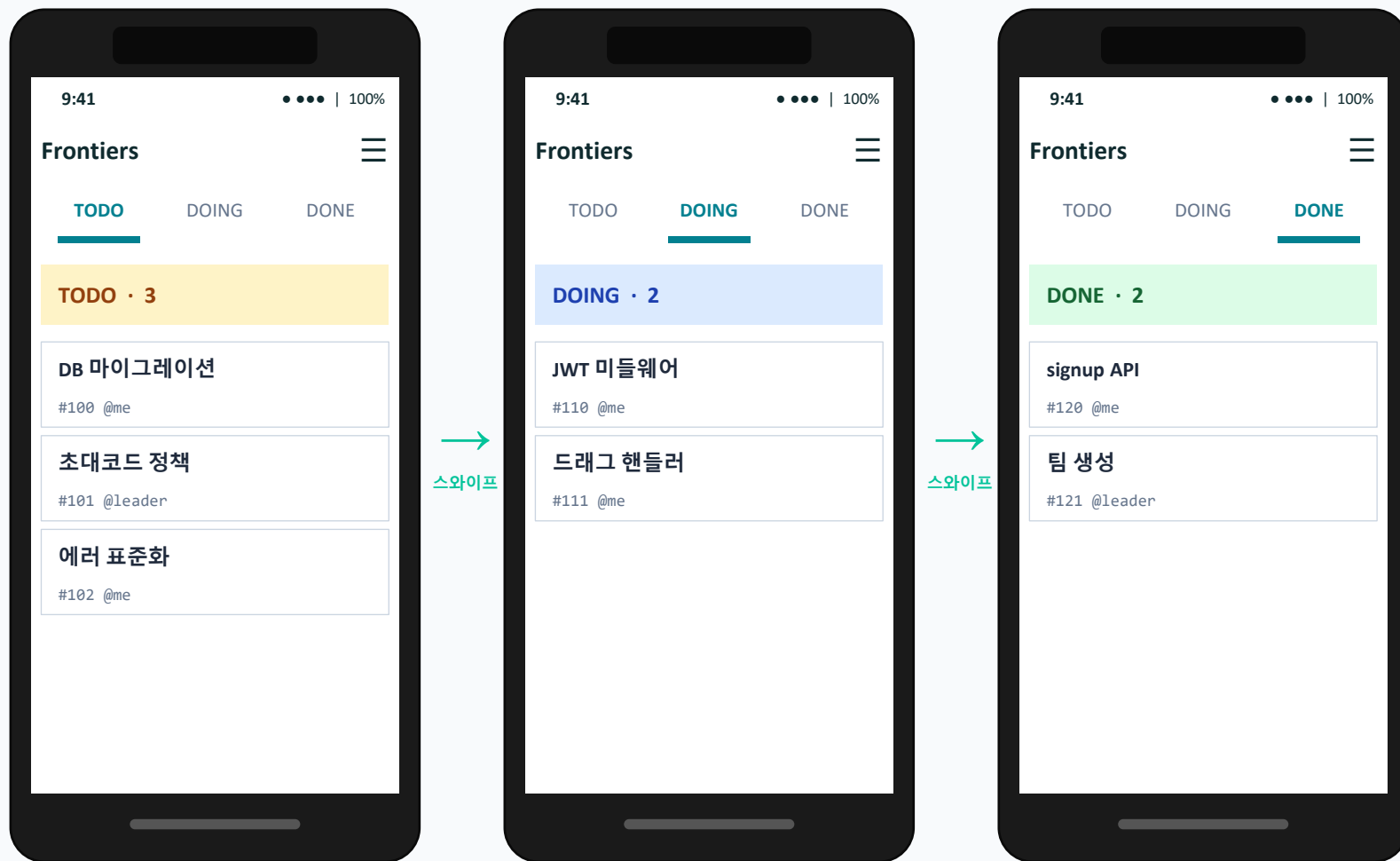
- 1 Exponential backoff: 5s → 10s → 20s → 40s → 60s 고정 (최대값)
- 2 재연결 시 since= 파라미터로 누락 메시지 일괄 수신
- 3 복구 시 토스트: '연결되었습니다' 사용자 입력은 큐에서 자동 전송

✓ Metric 보장

메시지 누락 0건
 = POST가 성공한(201) 메시지는 이후 모든 GET에서 노출.
 네트워크 끊김 동안 since= 파라미터로 재동기화 시 누락 없이 받음.
 DELETE된 메시지는 누락이 아님(다른 동작).

모바일 칸반 — 컬럼 스와이프

데스크탑 3컬럼 → 모바일 1컬럼씩 가로 스와이프. 헤더에 컬럼 인디케이터



모바일 대응

- 1 Breakpoint < 768px → 1컬럼 모드
- 2 좌우 스와이프로
TODO ↔ DOING ↔ DONE
- 3 상단 인디케이터로 현재 컬럼 표시
- 4 카드 길게 누르기 → 상태 변경 메뉴
(드래그 대신 메뉴 선택)
- 5 동일 API 사용 (반응형 UI만)
- 6 FAB 버튼 (+) 우하단 — 카드 추가

모바일 채팅 — 풀스크린 + 키보드

데스크탑 채팅 패널 → 모바일 풀스크린. 키보드 올라올 때 메시지 영역 축소

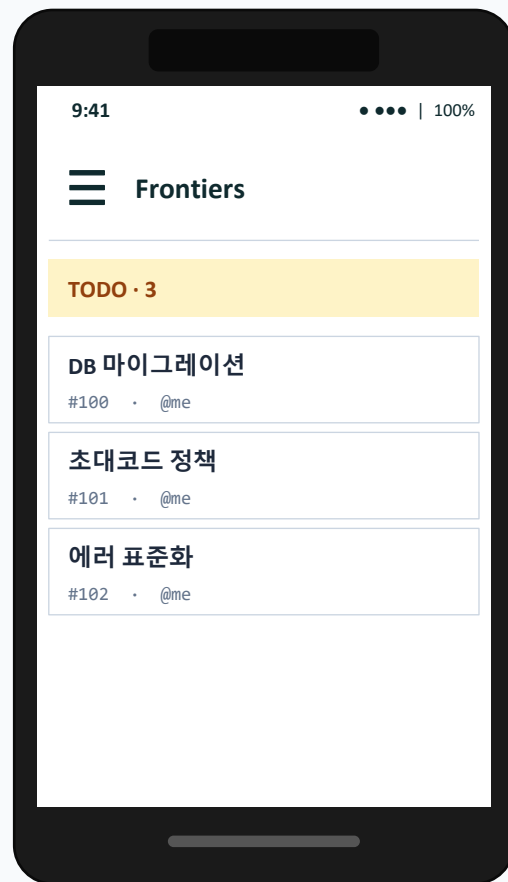


모바일 채팅 UX

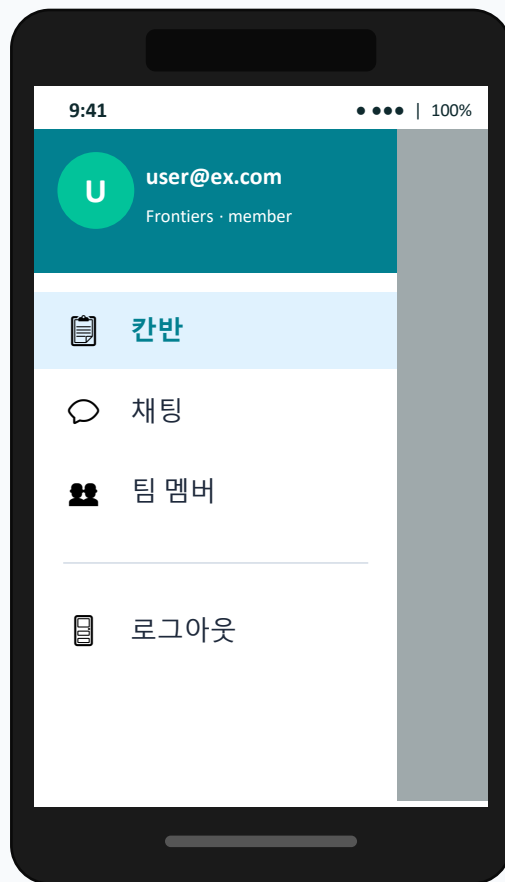
- 1 키보드 활성화 시 메시지 영역 자동 축소 (visualViewport API 사용)
- 2 전송 후 자동 스크롤 하단 고정
- 3 입력 포커스 시 폴링 5초 → 2초로 단축
- 4 꾸욱 누르기 → 본인 메시지면 삭제 메뉴
- 5 동일 API 사용:
POST /teams/{id}/messages
GET /teams/{id}/messages?since=
- 6 Pull-to-refresh로 강제 동기화

모바일 메뉴 — 햄버거 슬라이드

데스크탑 헤더의 팀 정보·로그아웃 → 모바일은 햄버거 메뉴로 통합



닫힘 (기본)



열림 (≡ 탭)

메뉴 항목 ↔ 데스크탑 매핑

- | | | |
|--------|---|------------------|
| ☰ 칸반 | ↔ | 헤더 [칸반] 탭 |
| 💬 채팅 | ↔ | 헤더 [채팅] 탭 |
| 👥 팀 멤버 | ↔ | 사이드 패널 |
| 🚪 로그아웃 | ↔ | 헤더 우측 이메일 → 로그아웃 |

i Breakpoint

< 768px: 햄버거 + 풀스크린 · 768~1024px: 헤더 통합 · > 1024px: 사이드 패널

ER 다이어그램 — 4 테이블

결정 #1, #4 반영: users.team_id, tasks.assignee_id 추가. ★ 표시는 통합본 변경점

users		★ 결정 #1
id	PK	
email	UNIQUE NOT NULL	
password_hash	NOT NULL	
★ team_id	FK→teams NULL	
created_at	TIMESTAMP	

N : 1

1 : 1
(owner)

teams	
id	PK
name	NOT NULL (1-30자)
invite_code	UNIQUE 'AAAA-9999'
owner_id	FK→users NOT NULL
created_at	TIMESTAMP

tasks		★ 결정 #4
id	PK	
team_id	FK→teams NOT NULL	
title	NOT NULL (1-100자)	
status	TODO DOING DONE	
creator_id	FK→users NOT NULL	
★ assignee_id	FK→users NULL	
★ created_at	TIMESTAMP (정렬)	

messages	
id	PK
team_id	FK→teams NOT NULL
user_id	FK→users NOT NULL
content	NOT NULL (1-1000자)
created_at	TIMESTAMP

인덱스 & 관계

- 1 tasks(team_id, created_at)
— 칸반 정렬용
- 2 messages(team_id, created_at)
— 채팅 폴링 (since=)
- 3 teams(invite_code) UNIQUE
— 합류 검증 O(1)
- 4 users.team_id 인덱스
— '내 팀' 빠른 조회
- 5 관계: tasks/messages N:1 → teams
users N:1 → teams (team_id)
teams 1:1 → users (owner)

★ 변경점: users.team_id (결정 #1, nullable, 1인 1팀) · tasks.assignee_id (결정 #4, nullable, '내 태스크' 정의) · tasks.created_at (정렬용)

API 18개 — 그룹별 카드

Auth 4 + Team 5 + Task 6 + Chat 3 = 18개. 색상으로 그룹 구분

Auth

4개

POST	/auth/signup	이메일+비번 → 201 + JWT
POST	/auth/login	→ 200 + JWT (24h)
POST	/auth/logout	stateless, 200만
GET	/auth/me	현재 사용자 정보

Team

5개

POST	/teams	팀 생성 + 초대코드 발급
POST	/teams/join	초대코드로 합류
GET	/teams/{id}	팀 정보 (결정 #8 추가)
GET	/teams/{id}/members	멤버 목록
DELETE	/teams/{id}/leave	팀 떠나기 (자기 자신)

Task

6개

GET	/teams/{id}/tasks	칸반 (filter: @me/미할당)
POST	/teams/{id}/tasks	카드 생성
GET	/tasks/{id}	단일 카드 상세
PUT	/tasks/{id}	제목·assignee 수정
PATCH	/tasks/{id}/status	★ 결정 #3 분리
DELETE	/tasks/{id}	creator OR owner만

Chat

3개

GET	/teams/{id}/messages	?since= 폴링 지원
POST	/teams/{id}/messages	1000자 이내
DELETE	/messages/{id}	본인만 (결정 #6)

ACME — Assumptions

가정 — propose 단계에서 묻지 않고 진행할 전제 조건

A

Assumptions

가정 — propose 단계에서 묻지 않고 진행할 전제 조건

핵심 항목

- ★ 1인 1팀 — 사용자는 동시에 하나의 팀에만 소속. team_id NULL = 미가입.
- 단일 언어 — 한국어 UI만. i18n은 Day 2 범위 외.
- 단일 시간대 — 서버·클라이언트 모두 KST. UTC 변환 X.
- 동시 접속자 5명/팀 이하 — 팀 최대 5명 기준 (그 이상은 추후).
- 이메일 인증 생략 — POST /auth/signup 즉시 계정 활성화.
- 모던 브라우저 가정 — Chrome/Safari 최신 2버전. IE 지원 X.

관련 결정: #1 (users.team_id)

ACME — Constraints

제약 — 반드시 지켜야 하는 시스템·정책 규칙



Constraints

제약 — 반드시 지켜야 하는 시스템·정책 규칙

핵심 항목

- ★ **JWT + 멤버십 검증** — 모든 `/teams/*` 라우트는 JWT + `user.team_id` 확인. 비멤버 → 403.
- ★ **DELETE `/tasks` 권한** — creator 또는 team owner만. 그 외 → 403.
- ★ **DELETE `/messages` 권한** — 본인만. owner도 타인 메시지 삭제 불가.
- ★ **logout stateless** — JWT 블랙리스트 없음. 200만 반환.
- **1팀당 1초대코드** — 고정. 재발급 Day 2 범위 외.
- **메시지 1000자 이내** — 클라+서버 양쪽 검증. 400 BAD_REQUEST.
- **JWT 만료 24시간** — 갱신 토큰 없음. 만료 시 로그인 재요청.

관련 결정: #5 (logout) · #6 (권한)

ACME — Metrics

측정 지표 — 검증·합격 기준



Metrics

측정 지표 — 검증·합격 기준

핵심 항목

- ★ 메시지 누락 0건 — POST 성공(201) 메시지는 이후 GET에서 100% 노출. DELETE는 누락 아님.
- 기능 5종 작동 — 회원가입/로그인/팀/칸반/채팅 모두 정상 흐름 통과 (정성).
- 에러 응답 표준 100% — 모든 4xx/5xx 응답이 { error: { code, message } } 형태.
- 신규 합류자 컨텍스트 1분 — 회원가입 → 칸반 진입까지 1분 이내 (정성 검증, 결정 #7).
- 칸반 드래그 < 50ms — 정성 검증으로 변경. 자동 측정 도구 미사용 (결정 #7).
- 권한 격리 100% — 비멤버가 /teams/{other_id}/* 접근 → 모두 403.

관련 결정: #6 (권한) · #7 (정성 검증)

ACME — Examples

예시 — 모호한 정의를 구체화하는 케이스

E

Examples

예시 — 모호한 정의를 구체화하는 케이스

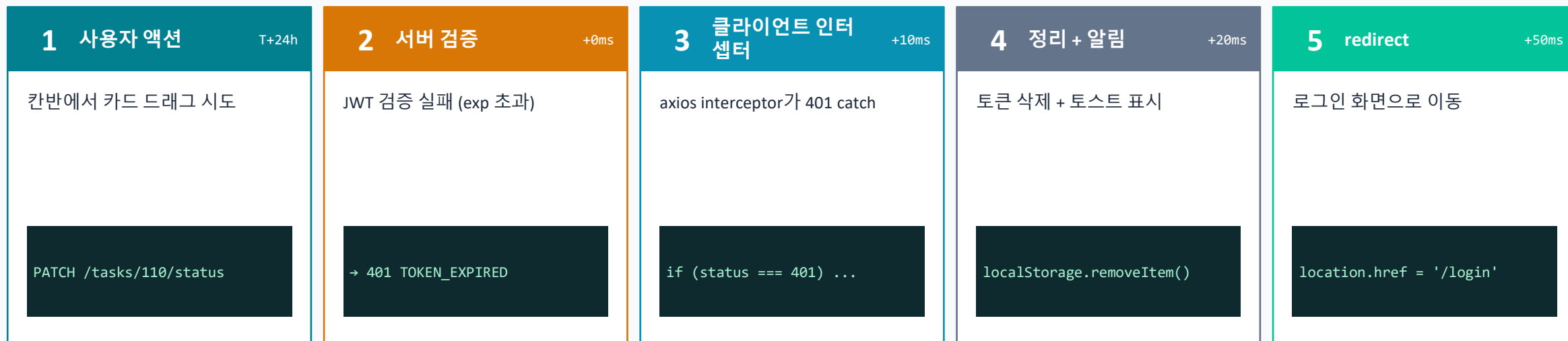
핵심 항목

- ★ '내 태스크' 정의 — WHERE tasks.assignee_id = current_user_id. creator_id 아님. (결정 #4)
- ★ 비멤버 접근 — user A (team=1), GET /teams/2/tasks → 403 { code: 'FORBIDDEN' }.
- 초대코드 형식 — 정규식 `^[A-Z]{4}-[0-9]{4}$`. 예: FRNT-2026. 클라+서버 양쪽 검증.
- 팀 소속 안 됨 — user.team_id = NULL인 사용자는 칸반/채팅 진입 불가. 팀 선택 강제.
- 신규 합류자 시나리오 — 회원가입 → 초대코드 입력 → 합류 → 칸반(스크롤) → 채팅(스크롤). 5분 이내.
- 토큰 만료 — GET /tasks 응답 401 → localStorage 삭제 → /login redirect. 직전 URL 저장 X.

관련 결정: #2 (스크롤) · #4 (assignee) · #6 (권한)

JWT 만료 흐름 — 자동 redirect

JWT 24h 만료 후 API 호출 시 401 → localStorage 삭제 → /login 강제 이동



HTTP 401 Unauthorized

```
{
  "error": {
    "code": "TOKEN_EXPIRED",
    "message": "인증이 만료되었습니다"
  }
}
```

⚠ 설계 결정 (결정 #5와 연관)

갱신 토큰 x — Day 2 범위 외. 만료 시 무조건 재로그인.
 직전 URL 저장 x — 로그인 후 칸반 기본 화면. 복귀 x.
 안 본 페이지 보호 — PWA route guard로 토큰 없으면 즉시 redirect.

에러 응답 표준 — HTTP 4xx / 5xx

모든 에러는 동일 형태: { error: { code, message, ...meta } }

표준 응답 형태

```
{
  "error": {
    "code": "<MACHINE_READABLE>",
    "message": "<사용자에게 보일 한국어>",
    "meta": { /* 옵션, code별 다른 */ }
  }
}
```

원칙

- ① code는 SCREAMING_SNAKE (기계 친화)
- ② message는 한국어 (사용자에게 그대로 표시)
- ③ meta는 옵션 (limit/actual 등 추가 컨텍스트)
- ④ 보안: 민감정보 노출 금지 (예: 이메일 존재 여부)

케이스별 표 (이 MVP 범위)

HTTP	code	발생 조건	message 예시
400	VALIDATION_ERROR	필드 형식 오류 (이메일·길이·필수)	올바른 형식이 아닙니다
400	TOO_LONG	메시지 1000자 초과 (POST /messages)	1000자 이내로 입력하세요
401	INVALID_CREDENTIALS	로그인 실패 (이메일 노출 x)	이메일 또는 비밀번호가 일치하지 않습니다
401	TOKEN_EXPIRED	JWT 만료 또는 누락	인증이 만료되었습니다
403	FORBIDDEN	비멤버가 /teams/{id}/* 접근	권한이 없습니다
403	NOT_OWNER	타인 메시지 삭제 시도	본인의 메시지만 삭제할 수 있습니다
404	NOT_FOUND	초대코드/리소스 없음	해당 항목을 찾을 수 없습니다
409	EMAIL_TAKEN	회원가입 시 중복 이메일	이미 가입된 이메일입니다

환경 분리 — 로컬 개발 / 운영 배포

로컬: FastAPI + SQLite 파일. 배포: Vercel (프론트 + 백엔드) + Neon (DB)

로컬 (개발)

백엔드

FastAPI 1개 서버

```
uvicorn main:app --reload
```

DB

SQLite 파일

```
sqlite:///./taskflow.db
```

프론트

정적 파일 직접 열기

```
live-server 또는 python -m http.server
```

↕ DATABASE_URL 환경변수로만 전환 ↕

운영 (배포)

GitHub

main push

```
git push origin main
```

Vercel

프론트 + 백엔드

정적 파일 + Serverless Functions

Neon

운영 PostgreSQL

```
postgres://...neon.tech
```

백엔드는 FastAPI를 Vercel Serverless Functions로 배포. main push 시 모두 자동 배포. DATABASE_URL 만 환경별로 다름.

통합 매핑표 — 화면 × API × DB

마지막 슬라이드. 모든 화면이 어떤 API와 DB 컬럼을 건드리는지 한눈에

화면	주 액션	API	DB 영향
회원가입	이메일+비번	POST /auth/signup	users INSERT
로그인	인증	POST /auth/login	users SELECT (검증만)
로그아웃	토큰 폐기	POST /auth/logout (옵션)	– (stateless)
팀 선택	팀 만들기	POST /teams	teams INSERT, users.team_id UPDATE
팀 선택	초대코드 합류	POST /teams/join	users.team_id UPDATE
멤버 목록	조회	GET /teams/{id}/members	users SELECT (team_id=?)
칸반	조회 + 필터	GET /teams/{id}/tasks	tasks SELECT
칸반	카드 생성	POST /teams/{id}/tasks	tasks INSERT
칸반	★ 상태 변경 (드래그)	PATCH /tasks/{id}/status	tasks UPDATE (status)
칸반	상세 수정	PUT /tasks/{id}	tasks UPDATE (title, assignee_id)
칸반	삭제	DELETE /tasks/{id}	tasks DELETE (권한 검증)
채팅	조회 (폴링)	GET /teams/{id}/messages?since=	messages SELECT
채팅	메시지 전송	POST /teams/{id}/messages	messages INSERT
채팅	메시지 삭제	DELETE /messages/{id}	messages DELETE (본인만)

📦 다음 단계 → 이 스토리보드를 클로드코드 /opsx:propose 입력으로 사용

총 화면 9종 · API 18개 · DB 4테이블 · 결정 8건 반영 · 스토리보드 42슬라이드

백엔드 스택 — FastAPI + PostgreSQL

로컬: SQLite 파일. 운영: Neon + Vercel. 나머지(라이브러리·구조·테스트)는 클로드코드 판단



백엔드 책임 (무엇을 만드는가)

- ① **API 18개 구현**
Auth 4 + Team 5 + Task 6 + Chat 3
- ② **JWT 발급 · 검증**
24h 만료, stateless (블랙리스트 x)
- ③ **비밀번호 해싱 + Validation**
이메일·길이·1000자·형식 모두 서버측 검증
- ④ **권한 검증 미들웨어**
team_id 멤버십, creator/owner, 본인만
- ⑤ **DB 4테이블 + 인덱스**
users·teams·tasks·messages, FK CASCADE
- ⑥ **에러 응답 표준화**
{ error: { code, message } } 일관 형태
- ⑦ **환경 분리 (로컬 / 운영)**
DATABASE_URL로 로컬 SQLite ↔ 운영 Neon 전환
- ⑧ **Vercel 배포 (운영)**
로컬은 uvicorn 직접 실행, 운영은 Vercel Serverless Functions

프론트엔드 스택 — Vanilla JS + Tailwind

사용자가 결정한 스택. 프레임워크 없음. 나머지는 클로드코드 판단

Vanilla JS

프레임워크 X · 순수 ES6+ · fetch API

+

Tailwind CSS

유틸리티 CSS · 반응형 breakpoint

프론트엔드 책임 (무엇을 만드는가)

- ① 9개 화면 구현 (HTML 페이지)
로그인·가입·팀선택·칸반·채팅·멤버...
- ② fetch로 API 18개 호출
JWT 헤더 자동 첨부, 401 catch 재로그인
- ③ JWT localStorage 관리
저장·읽기·삭제 + 페이지 진입 시 검사
- ④ Validation (클라측)
1차 검증 + 1000자 카운터 + 형식 체크
- ⑤ Drag & Drop (칸반)
HTML5 native drag API, drop 시 PATCH 호출
- ⑥ 채팅 폴링
setInterval 5초 + since= 파라미터로 증분
- ⑦ 반응형 UI (Tailwind)
md:768 / lg:1024 breakpoint, 모바일 햄버거